

Simulacro de Examen POO: Gestión de la Liga Pokémon

La Asociación Pokémon quiere gestionar el rendimiento de los entrenadores registrados para decidir quiénes son aptos para competir en la Liga.

Cada entrenador tiene un registro de los combates ganados durante los 5 días laborables de la semana, divididos en dos formatos de combate.

Se pide desarrollar el sistema completo siguiendo **estrictamente** estas indicaciones:

1. Enumerado Base

Crea un enumerado llamado Rango con los siguientes valores exactos: NOVATO, LIDER_GIMNASIO, ALTO_MANDO, CAMPEON.

2. Interfaz

Crea una interfaz llamada Evaluable que contenga la firma de un único método: boolean esAptoParaLiga();

3. Clase Abstracta Entrenador

Crea una clase abstracta llamada Entrenador que implemente la interfaz Evaluable.

Atributos obligatorios (usa la visibilidad adecuada):

- String id
- String nombre
- Rango rango
- int[][] registroVictorias

Sobre la matriz registroVictorias:

- Siempre tendrá un tamaño de **5 filas x 2 columnas** (5 \times 2).
- Las filas (0 a 4) representan los días de lunes a viernes.
- La columna 0 representa victorias en **Combates Individuales**.
- La columna 1 representa victorias en **Combates Dobles**.

Métodos obligatorios:

1. **Constructor con parámetros:** Recibe id, nombre, rango y la matriz registroVictorias, y los inicializa.
2. **Getters** para todos los atributos.
3. **toString():** Devuelve un String con el id, nombre y rango del entrenador.
4. **int calcularVictoriasSemanales():** Recorre toda la matriz bidimensional y devuelve la suma total de todas las victorias de la semana.
5. **int calcularVictoriasDia(int dia):** Recibe por parámetro un índice de fila (0 a 4) y devuelve la suma de las victorias (Individual + Doble) solo de ese día.
6. **Método abstracto double calcularPuntuacion().**
7. **void mostrarRegistroCombates():** Muestra por consola la matriz completa en forma de tabla o listado claro (ej. "Día 0 - Individuales: 3, Dobles: 1").

4. Subclases

a) Clase EntrenadorEstratega

- Hereda de Entrenador.
- **Atributo propio:** int medallasConseguidas.
- **Implementa:**
 - Constructor con parámetros (recibe los del padre + el suyo propio).
 - toString() sobrescrito (añade las medallas).
 - calcularPuntuacion(): Retorna un double usando la fórmula: (Victorias Totales de la semana * 1.5) + (medallasConseguidas * 10).
 - esAptoParaLiga(): Devolverá true si su puntuación calculada es mayor o igual a **100**.

b) Clase EntrenadorCriador

- Hereda de Entrenador.
- **Atributo propio:** int huevosEclosionados.
- **Implementa:**
 - Constructor con parámetros.
 - toString() sobrescrito (añade los huevos).
 - calcularPuntuacion(): Retorna un double usando la fórmula: (Victorias Totales de la semana * 1.0) + (huevosEclosionados * 5).
 - esAptoParaLiga(): Devolverá true si su puntuación calculada es mayor o igual a **80**.

5. Programa Principal (Main)

En la clase Principal, implementa los siguientes pasos en orden dentro del main:

a) Creación de objetos:

- Crea un array de tipo Entrenador con **4 posiciones**.
- Instancia y guarda en el array **2 objetos EntrenadorEstratega y 2 objetos EntrenadorCriador**.

b) Inicialización manual de matrices:

- Define manualmente las matrices de 5x2 para cada entrenador antes de pasarlas al constructor.
- *Ejemplo de matriz válida:*

```
int [][] victoriasAsh = {  
    {3, 1}, // Lunes (Fila 0): 3 Indiv, 1 Doble  
    {2, 0}, // Martes (Fila 1)  
    {4, 2}, // Miércoles (Fila 2)  
    {1, 1}, // Jueves (Fila 3)  
    {5, 3}  // Viernes (Fila 4)  
};
```

c) Procesamiento general (Recorrido del array): Recorre el array de entrenadores y, para cada uno, muestra:

1. Sus datos (usando toString()).
2. Su registro completo de combates (llamando a mostrarRegistroCombates()).
3. Sus victorias totales de la semana.
4. Su puntuación final.

5. Si es apto o no para la Liga Pokémon (texto descriptivo).

d) Estadísticas Finales (NUEVOS BUCLES): Al finalizar el recorrido anterior, debes calcular y mostrar por pantalla:

1. **El Entrenador con mayor puntuación:** Muestra el nombre y la puntuación del entrenador que haya sacado la nota más alta.
2. **Conteo de tipos:** Cuántos EntrenadorEstratega hay y cuántos EntrenadorCriador hay (usa instanceof).
3. **Medias por formato de combate (Columnas de la matriz):**
 - Calcula la media general de victorias en el formato **Individual** (columna 0) contando a TODOS los entrenadores.
 - Calcula la media general de victorias en el formato **Doble** (columna 1) contando a TODOS los entrenadores.
4. **El mejor día individual:** Encuentra qué entrenador ha conseguido más victorias **en un solo día** (usando el método calcularVictoriasDia()) e imprime su nombre y el número de victorias de ese día.